AS-BUILT PROJECT REPORT

Dukaan Saathi

A Hindi-first, voice + image inventory & *udhaar* (credit) assistant for a small kirana shop — built on a local Gemma-4-12B.

The shopkeeper just **talks in Hindi** or snaps a photo of a bill. Behind one clean screen, a local multimodal model reads it, decides what to do, updates two SQLite ledgers, and speaks the answer back — no typing, no cloud, no API bill.

Gemma-4-12B | llama.cpp | deepagents | Whisper STT | MMS-TTS | 2 × SQLite | Gradio | Slurm / CUDA

Complete current-state walkthrough · flows, architecture, databases, and verification · 2026-06-06

Contents

1 · Executive summary 3

2 · At a glance 4

3 · System architecture 6

4 · The model & how it is served 7

5 · The agentic layer (deepagents) 8

6 · The two databases 10

7 · Speech & vision 19

8 · Proactive agents 20

9 · End-to-end flow walkthroughs 21

10 · The Gradio app 22

11 · Deployment & operations 23

12 · Testing & verification 24

13 · Design decisions & gotchas 25

14 · Project layout 26

15 · Limitations & future work 27

16 · Appendix 28

1 Executive summary

A real kirana (corner) shop runs on a paper *bahi-khata*: stock, prices, expiry, and customer credit (*udhaar*) all live in a notebook the owner can lose, can't search, and can't analyse. The owner is fluent in Hindi but not in typing English or spreadsheets. **Dukaan Saathi** removes that notebook with a single voice-first screen.

The owner speaks (or types, or photographs a bill). A **local, multimodal Gemma-4-12B** — served by llama.cpp and driven by the **deepagents** framework — understands the Hindi, decides whether it is a change to record or a question to answer, calls the right tool (write or read), and replies concisely in spoken Hindi. Inventory lives in one SQLite database; sales and the credit ledger live in another. Nothing leaves the machine and there is no per-token API cost.

> **WHAT WORKS TODAY** Every capability below is implemented and verified end-to-end against the real 12B model: voice/text/image input, write & read tools, single- and cross-database lookups, a multi-step *“why isn’t this selling?”* diagnostic, bill OCR, three proactive agents, and a spoken-Hindi reply loop. The automated suite is **31 tests green**, and a ground-truth agent battery answered every task type correctly.

The numbers it runs on (live, from the seeded demo)

Catalog	**160** SKUs · 15 suppliers · 3,837 restock records (inventory.db)
Transactions	**6,152** sales over ~120 days · 35 customers · 100 udhaar entries (transactions.db)
Stock value	Rs 219,692 at cost → Rs 245,767 at MRP (potential margin Rs 26,075)
Today so far	Rs 10,117 across 29 sales
Outstanding udhaar	Rs 15,860 across 15 customers (15 overdue)

2 At a glance

What it does

Voice credit ledger

“Sharma ji ne 200 ka udhaar liya” / “kiska kitna baaki hai?” — add and query the khata entirely by voice.

Inventory + expiry

Track stock, MRP, purchase price and expiry; flag what’s about to expire.

Festival stock-up nudge

A 2026 festival calendar reminds the owner to restock before spikes.

“Why isn’t X selling?”

A multi-step diagnostic reasons over sales trend, stock, expiry and margin.

Reminder drafter

Drafts a polite Hindi WhatsApp message for each overdue udhaar.

Margin & stock value

Selling price, purchase price and MRP per item → live margin and stock value.

Tech stack

LLM + Vision / OCR	Gemma-4-12B (multimodal, Q8_0 GGUF + mmproj) via llama.cpp llama-server
Agentic framework	deepagents (LangChain) driving the local model through a ChatOpenAI instance
Speech → Text	faster-whisper large-v3 (Hindi, numpy in, no system ffmpeg)
Text → Speech	facebook/mms-tts-hin (open) + a Hindi number-words pass; Parler optional/gated
Databases	two SQLite files — inventory.db & transactions.db — unified for reads via ATTACH
Frontend	Gradio single-screen app (mic · photo · chat · today-dashboard · alerts)
Deployment	one Slurm GPU allocation (RTX A6000) runs llama-server + the app together

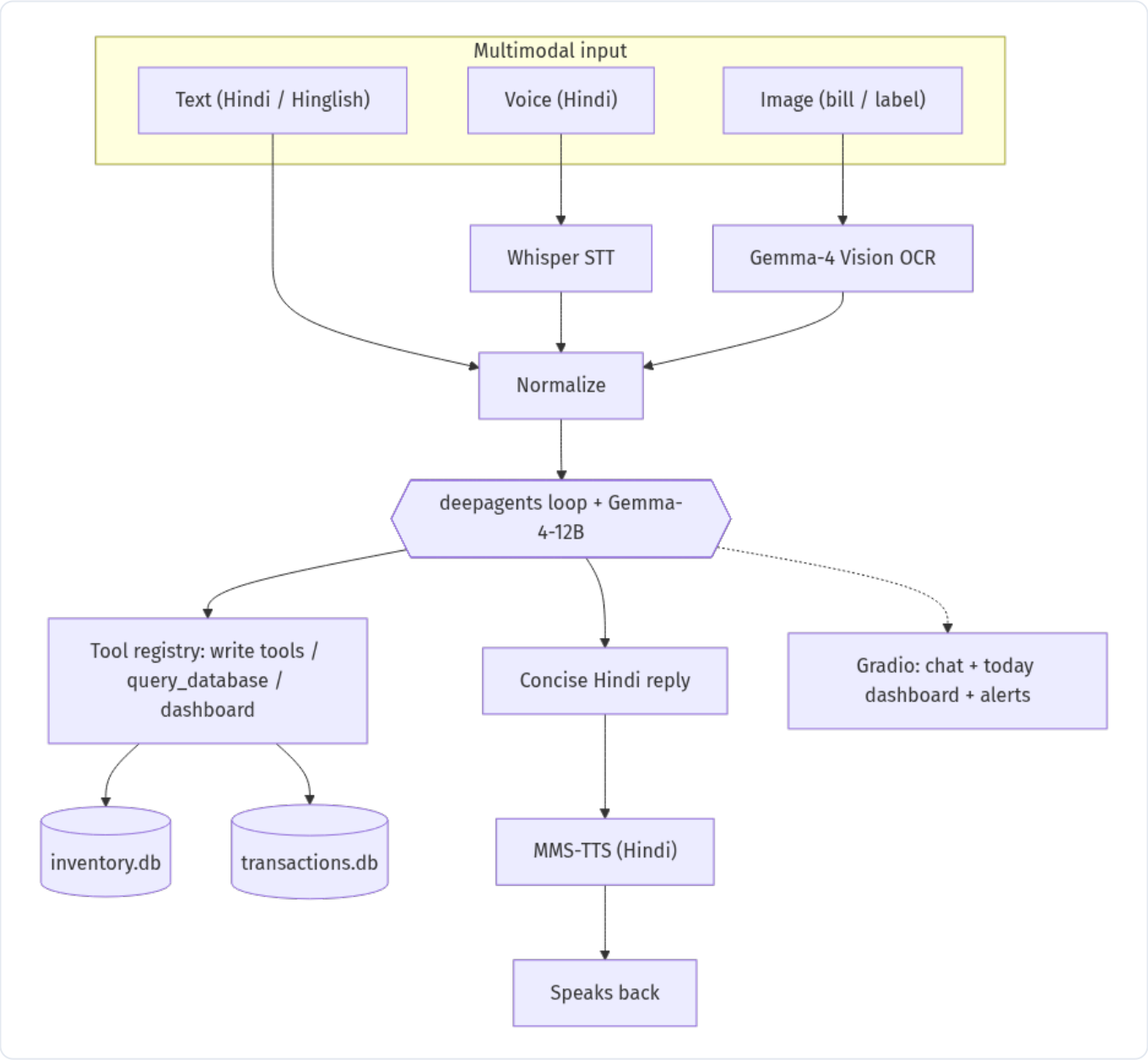
How the build differs from the original design

The original `design.md` sketched a single-database, Gemma-3 system. The as-built system evolved on several axes — this report documents the *current* reality:

Model	design.md said Gemma-3 → **Gemma-4-12B** (real, multimodal: text + image + audio)
Agent	“simple function-calling loop” → the **deepagents** framework
Database	one SQLite file → **two databases** (stock vs khata) with an ATTACH read layer
TTS	Indic Parler-TTS → **MMS-TTS** (Parler’s HF repo is gated)
Reasoning	added: Gemma-4 **thinking-mode disabled** for fast, non-empty replies

3 System architecture

One request flows left-to-right through five stages: **input** (voice, text or image), **normalisation** (speech→text, image→text), the **agent** (Gemma-4 deciding and calling tools), the **data layer** (two databases), and the **response** (a concise Hindi reply, optionally spoken). The same agent handles both directions: recording changes (writes) and answering questions (reads).



D1 — End-to-end system architecture: multimodal input → normalize → deepagents/Gemma-4 → two databases → spoken Hindi reply.

Three properties make it fit a “build small” brief: it is **local** (one GPU, no cloud), **multimodal in one model** (Gemma-4 reads both text and bill photos, so there is no separate OCR model), and **honest about scope** — a 12B model is genuinely enough for routing, text-to-SQL, short reasoning and drafting.

4 The model & how it is served

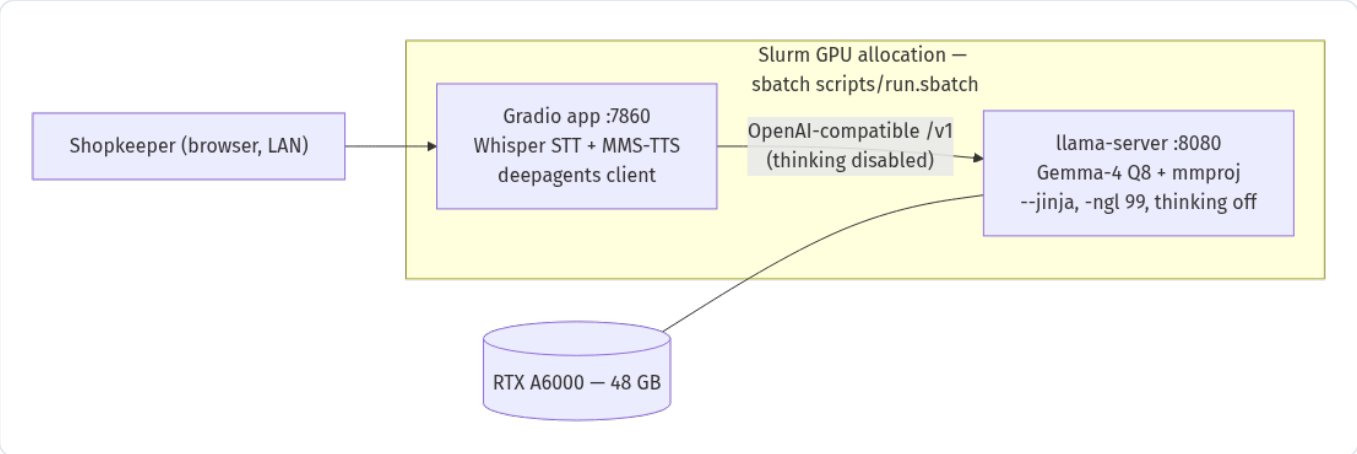
The brain is **Gemma-4-12B-it** in GGUF form (`ggml-org/gemma-4-12B-it-GGUF`), quantised to **Q8_0** (~12.7 GB, near-lossless) with its multimodal projector `mmproj` (~159 MB) for vision. It is served by a CUDA build of llama.cpp's `llama-server` , which exposes an OpenAI-compatible API.

Launch flags that matter

```
llama-server -m gemma-4-12B-it-Q8_0.gguf \
  --mmproj mmproj-gemma-4-12B-it-Q8_0.gguf \
  --host 127.0.0.1 --port 8080 -c 32768 -ngl 99 --jinja
```

--mmproj	loads the vision projector → the same endpoint accepts bill/label images
-ngl 99	offload all layers to the GPU (fully on the RTX A6000)
--jinja	use the model's chat template → enables OpenAI-style tool calling
-c 32768	32k context (Gemma-4 supports far more; 32k is ample here)

WHY THINKING-MODE IS TURNED OFF Gemma-4 ships a native "thinking" mode. Left on, a plain chat request returned an *empty* `content` (all tokens went to hidden reasoning) and took ~3.4 s. Sending `extra_body={"chat_template_kwargs": {"enable_thinking": false}}` on every request turns it off — the same prompt then answers in **0.2 s** with real text, and tool-calling stays reliable. The deep-agent loop supplies multi-step reasoning instead. This is wired centrally in `dukaan/llm.py` .

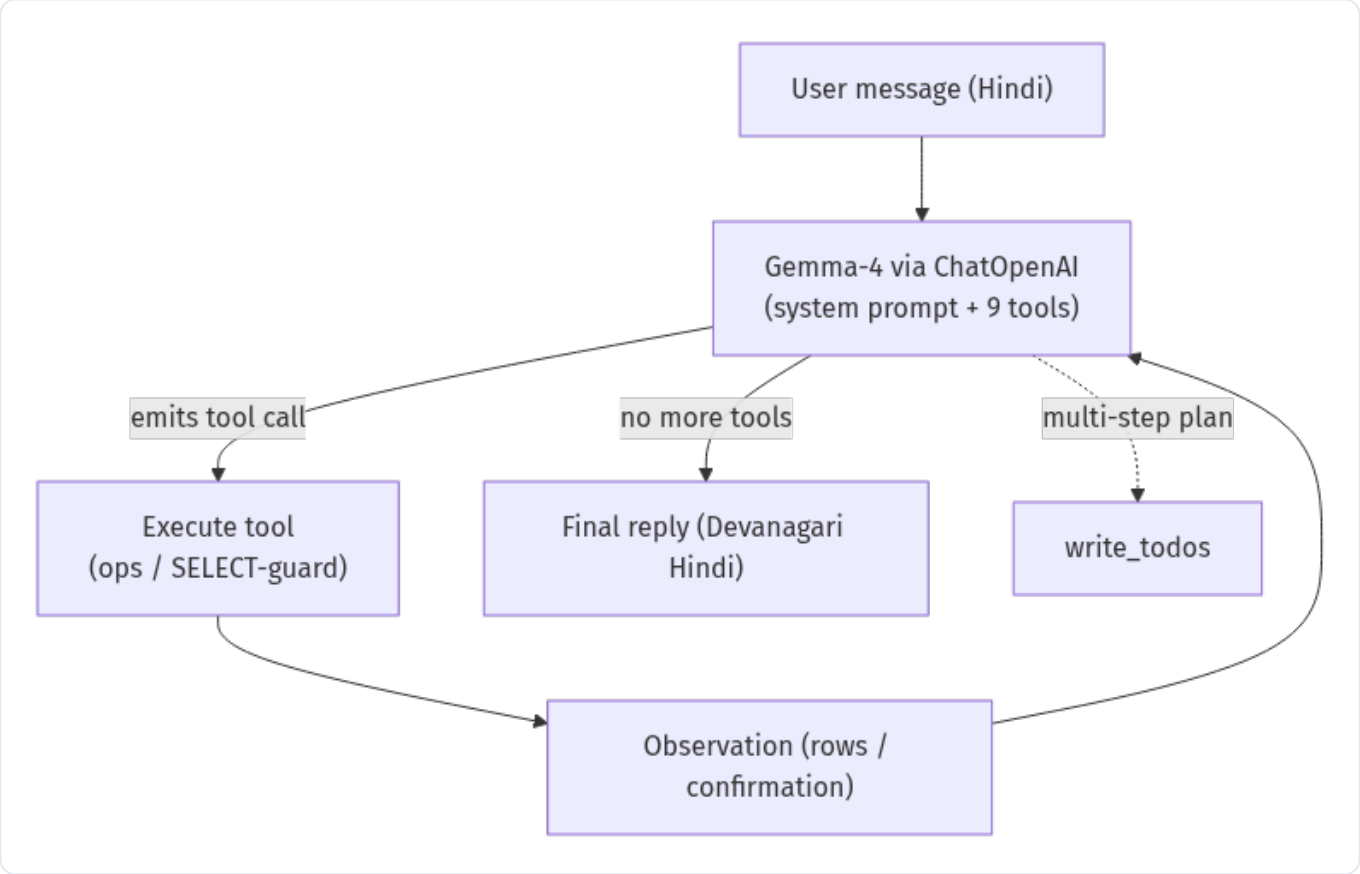


D2 — Serving: one Slurm GPU allocation runs llama-server (:8080) and the Gradio app (:7860) together.

The Python side never imports the model. `dukaan/llm.py` exposes a `ChatOpenAI` instance (for the agent) and a raw OpenAI client (for one-shot vision/normalise/draft calls) — both pointed at `http://127.0.0.1:8080/v1` .

5 The agentic layer (deepagents)

The agent is built with **deepagents 0.6.8** (LangChain's deep-agent framework). It is driven by the *local* model: we pass a `ChatOpenAI` **instance** (not an `"openai:..."` string — that would route to the OpenAI Responses API, which llama.cpp does not implement) into `create_deep_agent(...)`. No Anthropic key is needed.



D3 — The deepagents loop: the model reasons, calls tools, reads observations, and loops until it produces a final Hindi reply.

The tool registry (9 tools)

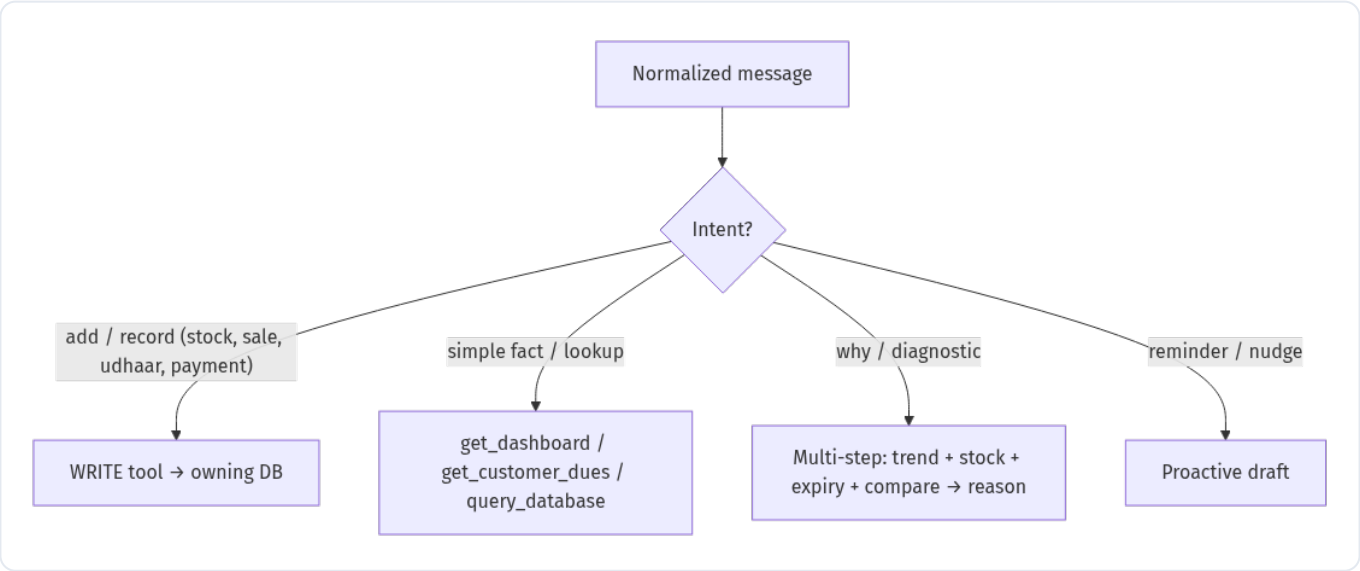
`add_inventory_tool`	add / create stock for an item
`record_sale_tool`	log a sale and decrement stock
`record_purchase_tool`	log a supplier restock and increase stock
`add_udhaar_tool`	add a credit (udhaar) entry for a customer
`record_payment_tool`	record a repayment against a customer's udhaar
`query_database`	run a read-only SELECT across both DBs (text-to-SQL)
`get_dashboard`	today's snapshot: stock value, sales, expiry, low stock, udhaar
`get_item_detail`	one item: stock, margin, expiry, 30-day sales

`get_customer_dues`	one customer’s balance, or the whole pending list

Each tool is a typed `@tool(parse_docstring=True)` function wrapping the pure-Python `dukaan/ops.py` layer. Write tools route to the owning database; the read tool `query_database` runs only through the hardened SELECT guard (see §6).

How it decides what to do

A Hindi-first system prompt (embedding the live DB schema) tells the model to classify each turn and act. This collapses the classic “router → single-turn / multi-turn” design into the agent’s own behaviour:



D4 — Intent routing, as encoded in the agent’s system prompt.

Memory	an `InMemorySaver` checkpointer keyed by a per-session `thread_id` → multi-turn context
Step budget	`recursion_limit = 60` (LangGraph default 25 is too low for multi-tool turns)
Subagents	deliberately none — deepagents 0.6.8 has an open recursion bug with subagents
Reply rule	always concise Devanagari Hindi; never mentions SQL/tools/steps; never touches the file tools

> **DIAGNOSTIC = EMERGENT MULTI-STEP** For “*X kyun nahi bik raha?*” the agent isn’t special-cased. It plans, calls `get_item_detail` and `query_database` several times (sales trend, stock, expiry, comparison), then reasons in plain Hindi and suggests an action (price cut, shelf placement). The depth is just the loop running longer.

6 The two databases

Data is split across **two SQLite files**, mirroring a real shop's mental model: the **supply side** (what's on the shelf) and the **money side** (who bought what, and who owes what).

inventory.db	**suppliers**, **inventory** (with expiry, MRP, purchase price, reorder level, HSN), **purchases** (restocks)
transactions.db	**customers**, **sales**, **ledger** (udhaar debits & repayments)

TWO FILES, BUT READS STILL JOIN ACROSS THEM Writes go to the owning file (`execute_inv` / `execute_txn`). For reads, `get_attached_ro_conn()` opens an in-memory connection and `ATTACH` -es both files *read-only* as `inv` and `txn` , so the agent can still `JOIN inv.inventory ⋈ txn.sales` in one query. The LLM-facing `run_select()` runs only on this attached connection.

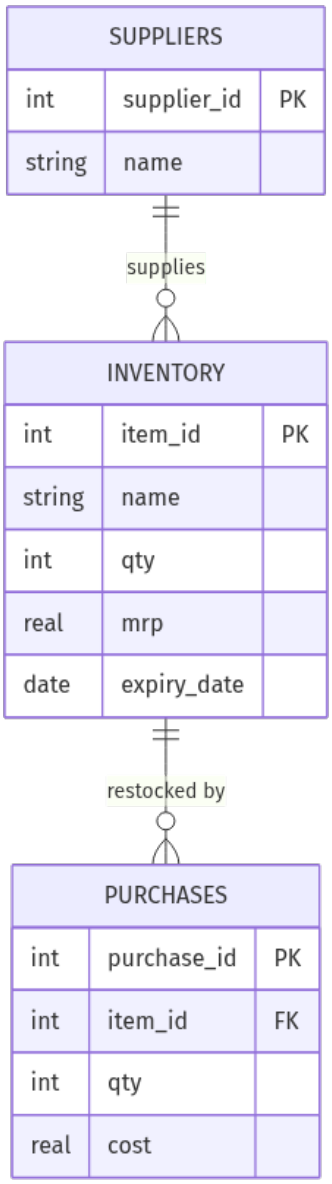
THE SELECT GUARD (SECURITY) All LLM-generated SQL passes `is_safe_select()` : it must be a single `SELECT` / `WITH` , no stacked statements, and no write/DDL keyword (`insert, update, delete, drop, attach, pragma…`). On top of that it executes on an OS-level read-only connection — two independent safeguards, so even a prompt-injected query cannot mutate or exfiltrate beyond a read.

Schemas (from `dukaan/db.py`)

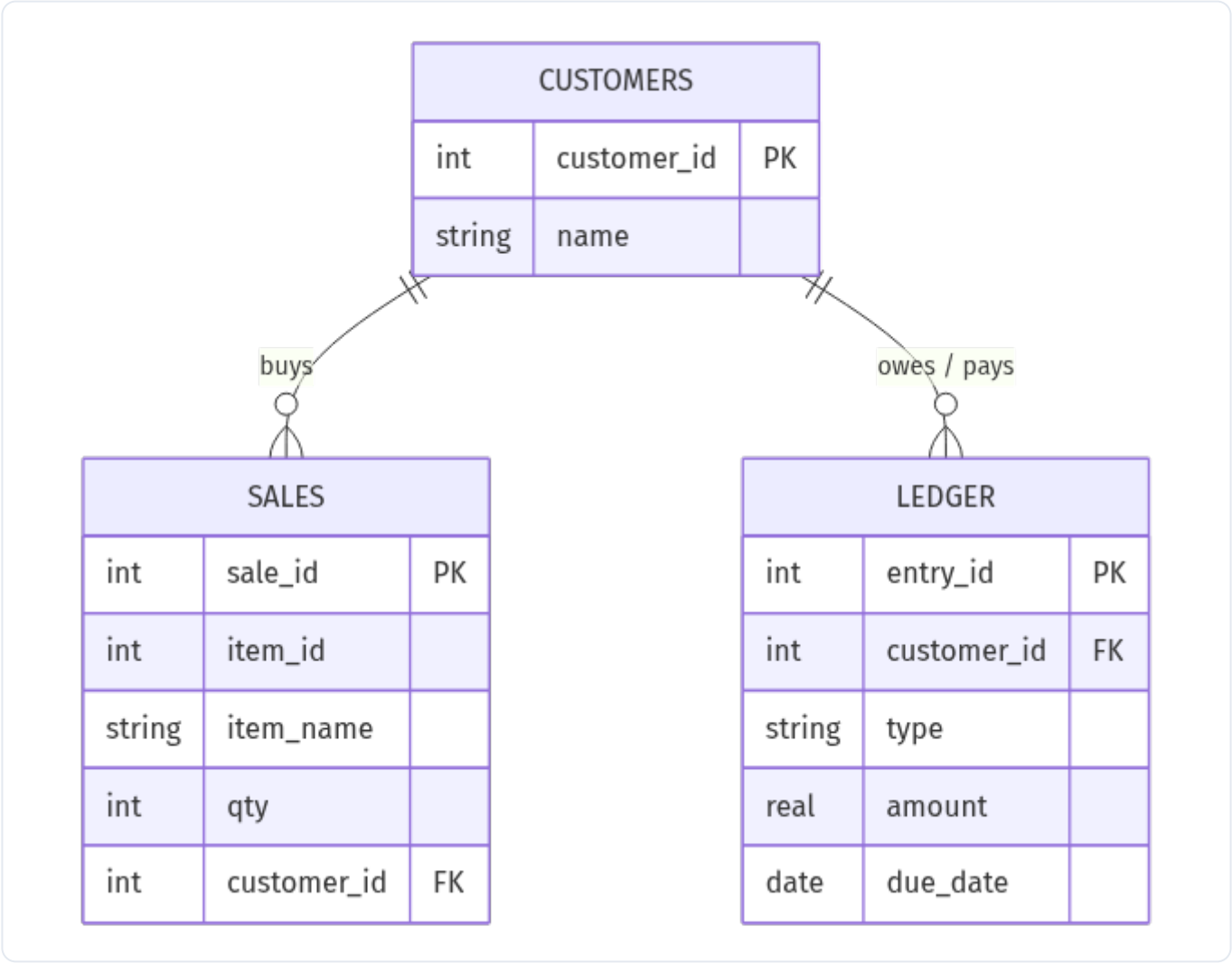
```
CREATE TABLE IF NOT EXISTS suppliers (
    supplier_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name        TEXT NOT NULL,
    phone       TEXT,
    focus       TEXT
);
CREATE TABLE IF NOT EXISTS inventory (
    item_id        INTEGER PRIMARY KEY AUTOINCREMENT,
    name           TEXT NOT NULL,
    category       TEXT,
    brand          TEXT,
    unit           TEXT,
    qty            INTEGER NOT NULL DEFAULT 0,
    mrp            REAL    NOT NULL DEFAULT 0,
    purchase_price REAL    NOT NULL DEFAULT 0,
    expiry_date    TEXT,
    reorder_level  INTEGER DEFAULT 0,
    hsn            TEXT,
    supplier       TEXT
);
CREATE TABLE IF NOT EXISTS purchases (
    purchase_id INTEGER PRIMARY KEY AUTOINCREMENT,
    item_id     INTEGER REFERENCES inventory(item_id),
    supplier    TEXT,
    qty         INTEGER NOT NULL,
    cost        REAL    NOT NULL,
    ts          TEXT    NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS customers (
    customer_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    name         TEXT NOT NULL,
    phone        TEXT,
    credit_limit REAL DEFAULT 0
);
CREATE TABLE IF NOT EXISTS sales (
    sale_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    item_id     INTEGER,
    item_name   TEXT,
    qty         INTEGER NOT NULL,
    sale_price  REAL    NOT NULL,
    ts          TEXT    NOT NULL,
    customer_id INTEGER REFERENCES customers(customer_id)
);
CREATE TABLE IF NOT EXISTS ledger (
    entry_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER REFERENCES customers(customer_id),
    type        TEXT NOT NULL,            -- 'debit' = udhaar taken, 'credit' = repayment
    amount      REAL NOT NULL,
    items       TEXT,
    due_date    TEXT,
    ts          TEXT NOT NULL
);
```

Entity-relationship diagrams



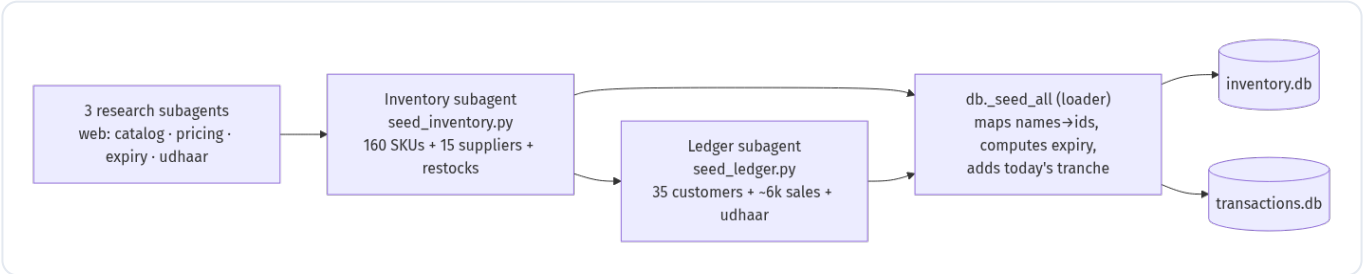
D5 — inventory.db (key fields shown; full columns in the schema above): suppliers → inventory → purchases.



D6 — transactions.db (key fields shown): customers → sales and customers → ledger (udhaar). sales.item_id is a cross-database reference to inv.inventory.

How the data was created

The demo data is not hand-typed. A small **agent team** built it: three web-research subagents studied real Indian kirana stores (catalog, brands, per-category margins, shelf life, udhaar and footfall patterns); an **inventory subagent** wrote `seed_inventory.py` (the catalog + a restock generator); a **ledger subagent** wrote `seed_ledger.py` (customers + a deterministic ~120-day sales & udhaar generator) consistent with that catalog. A loader (`db._seed_all`) maps item names → ids across the two files, derives realistic remaining expiry, and adds a small “today so far” tranche so the demo is never empty.



D7 — The data-generation pipeline: research → per-database subagents → loader → two .db files.

What the data actually looks like

Real rows sampled live from the two databases at build time:

inv.suppliers — sample

name	phone	focus
Sharma Distributors	98110 22451	ITC, Aashirvaad & Sunfeast — atta, biscuits, noodles, spices
Gupta Trading Co.	98100 73388	HUL — soaps, detergents, shampoo, tea (Surf, Lifebuoy, Brooke Bond)
Verma Agencies	99581 40927	Nestle — Maggi, KitKat, Cerelac, coffee, dairy whitener
Amul Parlour Depot	98184 61203	Amul — milk, butter, ghee, paneer, cheese, dahi (cold chain)
Mother Dairy Booth	98112 55719	Mother Dairy — token milk, dahi, lassi, paneer (daily)
Britannia Super Stockist	98715 30844	Britannia — bread, rusk, Good Day, Marie, Bourbon, cheese

inv.inventory — one item per category (showing the catalog’s breadth)

name	category	brand	unit	qty	mrp	purchase_price	expiry_date	reorder_level
Nestle Cerelac Wheat-Apple 300g	Baby Care	Cerelac	300g pack	9	Rs 280	Rs 252	2027-03-23	4
Parle-G Biscuit Rs5	Biscuits	Parle	55g pack	130	Rs 5	Rs 4.45	2026-10-31	60
Britannia White Bread 400g	Bread	Britannia	400g loaf	16	Rs 45	Rs 39.15	2026-06-09	12
Coca-Cola 750ml	Cold Drinks	Coca-Cola	750ml PET	48	Rs 40	Rs 36	2026-11-13	24
Cadbury Dairy Milk Rs10	Confectionery	Cadbury	13.2g bar	90	Rs 10	Rs 8.80	2026-10-08	40
Amul Gold Milk 500ml	Dairy	Amul	500ml pouch	40	Rs 34	Rs 31.62	2026-06-06	30
Almonds (Badam) loose	Dry Fruits	Local	250g pack	11	Rs 230	Rs 197.80	2026-12-06	5
Fortune Sunlite Sunflower Oil 1L	Edible Oil	Fortune	1L pouch	24	Rs 165	Rs 156.75	2026-08-12	10
Table Eggs (loose)	Eggs	Local	per egg	90	Rs 7	Rs 6.02	2026-06-07	60
Surf Excel Easy Wash 1kg	Home Care	Surf Excel	1kg pack	18	Rs 140	Rs 121.80	—	8
Maggi 2-Min Masala Noodles 70g	Instant Food	Maggi	70g pack	140	Rs 14	Rs 12.18	2026-07-03	60
Colgate Dental Cream 100g	Personal Care	Colgate	100g tube	26	Rs 76	Rs 64.60	2027-08-10	12

inv.purchases — recent restocks (joined to item names)

purchase_id	item	supplier	qty	cost	ts
3837	Maaza Mango 600ml	Hindustan Beverages	19	Rs 674.74	2026-06-06T16:05:00
3836	Amul Fresh Paneer 200g	Amul Parlour Depot	12	Rs 1,114.18	2026-06-06T14:20:00
3835	Amul Gold Milk 500ml	Amul Parlour Depot	30	Rs 955	2026-06-06T13:05:00
3834	Parle-G Biscuit Rs5	Patel Wholesale Mandi	155	Rs 666.55	2026-06-06T11:40:00
3833	Amul Gold Milk 1L	Amul Parlour Depot	29	Rs 1,837.92	2026-06-06T11:00:00
3832	Haldiram Aloo Bhujia 200g	Ganesh Provision Stores	23	Rs 1,048.87	2026-06-06T10:45:00

txn.customers — sample

name	phone	credit_limit
Sharma ji	98110 24517	Rs 5,000
Verma ji	98100 31882	Rs 4,000
Gupta aunty	99581 47203	Rs 3,500
Khan bhai	98184 60219	Rs 6,000
Lakshmi aunty	98112 55703	Rs 2,500
Reddy garu	98715 38044	Rs 5,000

txn.sales — most recent (joined to customer; many are walk-ins)

item_name	qty	sale_price	ts	customer
Tata Sampann Toor Dal 1kg	4	Rs 244	2026-06-06T08:58:00	—
Cadbury Dairy Milk Silk 60g	3	Rs 80	2026-06-06T08:56:00	—
Maaza Mango 600ml	4	Rs 40	2026-06-06T08:56:00	—
Horlicks Classic Malt 500g	5	Rs 280	2026-06-06T08:56:00	—
Kinley Water 20L Can	3	Rs 80	2026-06-06T08:53:00	—
Amul Gold Milk 500ml	5	Rs 34	2026-06-06T08:53:00	—
Bingo Mad Angles Rs20	2	Rs 20	2026-06-06T08:51:00	—
Brown Eggs 6-pack	5	Rs 72	2026-06-06T08:48:00	—

txn.ledger — udhaar entries (debit = taken, credit = repaid)

customer	type	amount	items	due_date
Mishra ji	debit	Rs 150	festival ka samaan	2026-02-22
Sengupta didi	debit	Rs 750	tel aur masala	2026-02-25
Reddy garu	debit	Rs 600	Maggi, biscuit, namkeen	2026-02-23
Gupta aunty	debit	Rs 300	ghee, paneer, dahi	2026-02-28
Mishra ji	credit	Rs 110	—	—
Ansari bhai	debit	Rs 500	atta, dal, tel	2026-02-26
Verma ji	debit	Rs 250	tel aur masala	2026-03-05
Reddy garu	credit	Rs 340	—	—

Catalog by category (inventory.db)

category	items	units in stock
Staples	15	358
Personal Care	12	389
Dairy	12	242
Spices	10	120
Biscuits	10	376
Snacks	9	360
Instant Food	9	299
Home Care	9	142
Edible Oil	9	117
Confectionery	9	765
Cold Drinks	9	330
Tea & Coffee	8	114
Stationery	6	161
Pooja	6	91
Dry Fruits	6	55
Tobacco	5	120
Baby Care	5	35
Water	4	142
Bread	4	43
Eggs	3	113

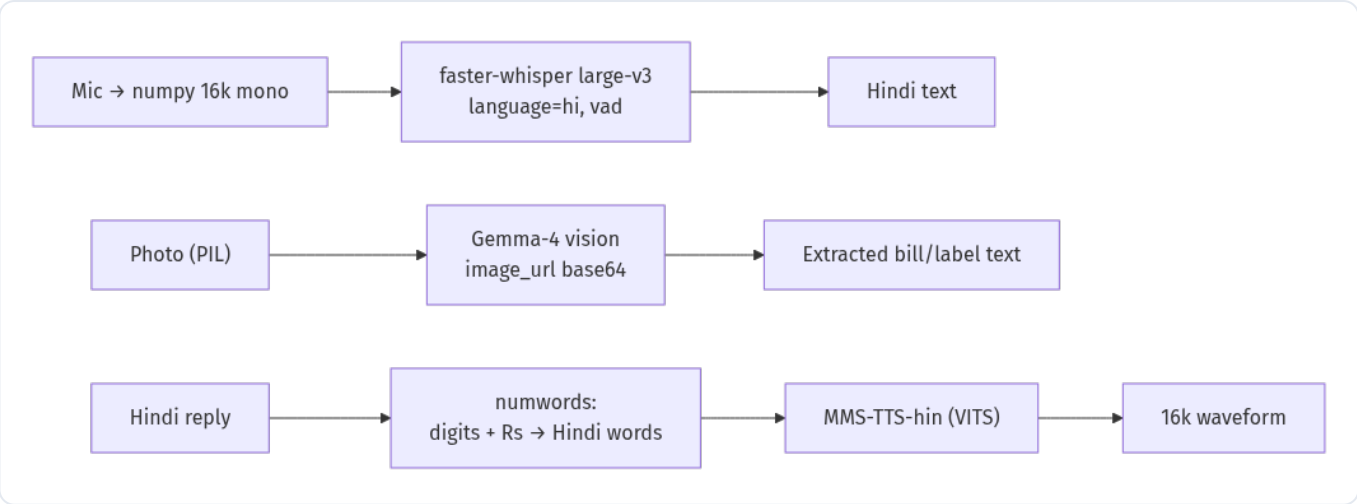
Top sellers (cross-database join: txn.sales ⋈ inv.inventory)

item	category	units sold
Table Eggs (loose)	Eggs	1538
Lay's Magic Masala Rs10	Snacks	360
Lay's Classic Salted Rs20	Snacks	339
Kurkure Masala Munch Rs20	Snacks	332
Bingo Mad Angles Rs20	Snacks	330
Kurkure Rs10	Snacks	324

CONSISTENCY CHECK All 6,152 sales reference real catalog items (0 orphans), udhaar balances reconcile to Rs 15,860 across 15 customers, and the cross-database top-seller join works — proving the ATTACH read layer in practice.

7 Speech & vision

The voice and image paths are deliberately light and ffmpeg-free.



D8 — Speech-to-text (Whisper), image OCR (Gemma-4 vision), and text-to-speech (MMS-TTS).

Speech → text

faster-whisper large-v3 transcribes Hindi. It accepts a numpy float32 array directly (Gradio's mic gives `(sr, ndarray)`), so there is **no system ffmpeg** dependency — audio is resampled to 16 kHz mono with `scipy.signal.resample_poly`.

Image → text (OCR)

Because Gemma-4 is multimodal, bill/label photos go to the *same* model via the OpenAI `image_url` (base64) content part — no separate OCR model. The extracted text is fed back to the agent as the user's effective message, which then calls the write tools.

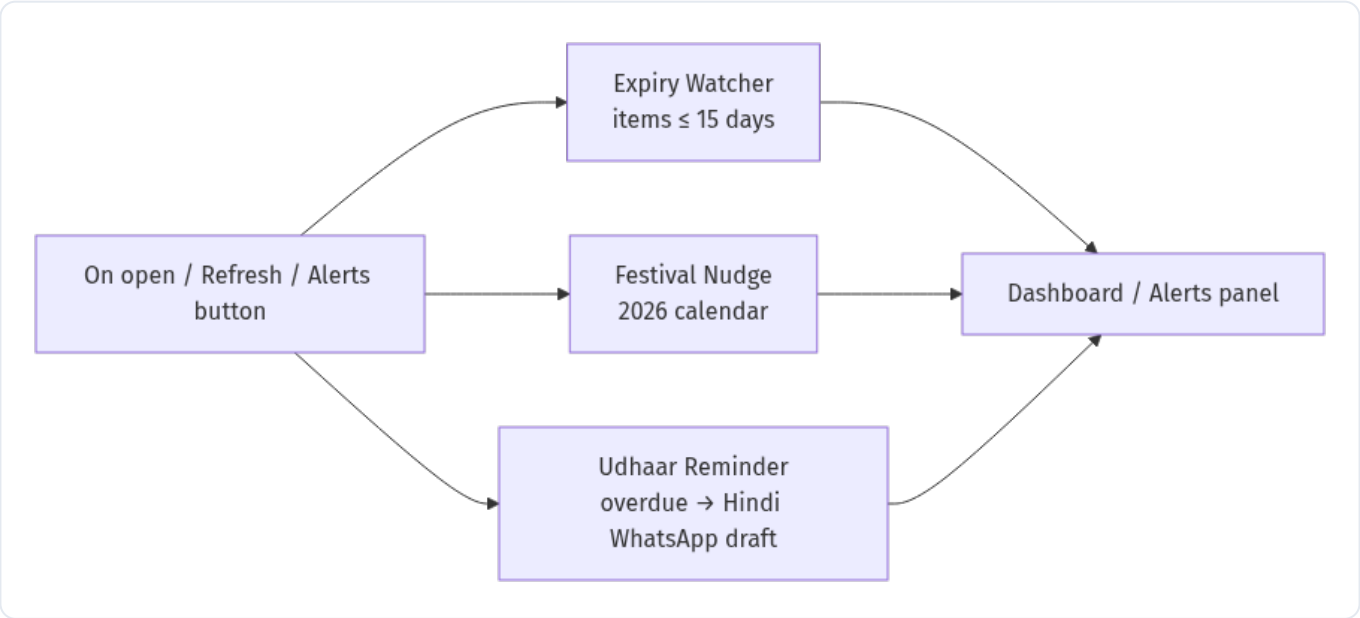
Text → speech

facebook/mms-tts-hin (open VITS) speaks the reply. Two fixes make it demo-ready:

> **MMS ONLY VOICES DEVANAGARI, AND CAN'T READ DIGITS** Romanised Hindi tokenises to almost nothing (garbled 0.4 s clips), so the agent is forced to reply in Devanagari. And MMS can't pronounce Latin digits — so `dukaan/numwords.py` converts numbers and Rs amounts to Hindi words before synthesis (e.g. "Rs 528" → "paanch sau atthaais rupaye"). Parler-TTS would sound better but its HF repo is gated, so MMS is the working default.

8 Proactive agents

Three agents surface things the owner didn’t ask about. They run on app open, on the dashboard *Refresh*, and behind an *Alerts* button (the LLM-drafted reminders run on demand so the dashboard stays instant).



D9 — Proactive agents feeding the dashboard and the alerts panel.

Expiry Watcher	lists items expiring within 15 days, with days-left, in friendly Hindi
Festival Nudge	a 2026 Indian festival calendar; if one is within 30 days, suggests what to stock up (cross-referenced with low / slow-moving items)
Udhaar Reminder	for each overdue customer, drafts a polite 1–2 line Hindi WhatsApp message (LLM, with a template fallback when the server is down)

RESILIENT BY DESIGN Every LLM call in the proactive layer is wrapped so a missing/slow server falls back to a fixed Hindi template — the dashboard never breaks.

9 End-to-end flow walkthroughs

Four representative turns, each tracing input → agent → data → reply.

A · Voice WRITE — “10 Parle-G ke packet aaye, 5 rupaye wala”

Mic → Whisper → Hindi text → agent recognises a restock → calls `record_purchase_tool(item, qty=10, …)` → a row is appended to `inv.purchases` and `inv.inventory.qty` grows → reply: “…maal darj kar diya, ab kul N packet hain.”

B · Cross-DB LOOKUP — “aaj kitni bikri hui?”

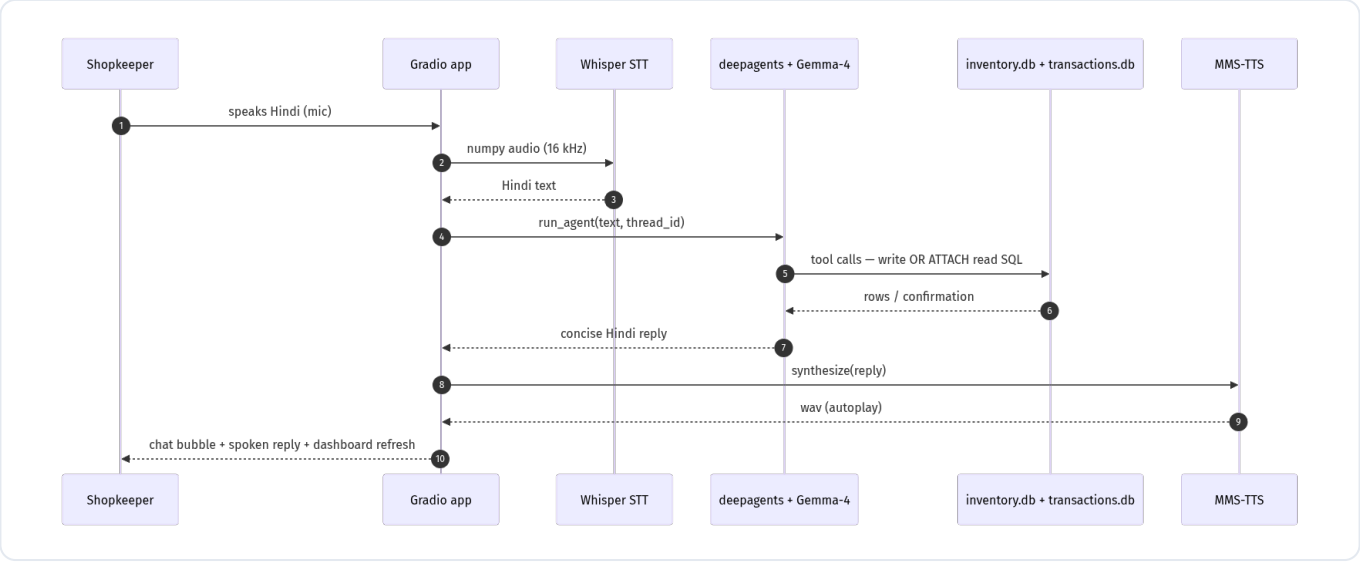
Agent calls `get_dashboard` (or writes a SELECT over `txn.sales`) → answers with today’s revenue, unit count and the top item — numbers that match `ops.today_summary()` exactly.

C · DIAGNOSTIC — “Maggi kyun nahi bik raha?”

Agent calls `get_item_detail` + `query_database` several times (30-day sales, stock, expiry, margin), then reasons in Hindi and suggests a combo offer or price check — a genuine multi-step investigation, not a single lookup.

D · IMAGE / OCR add — a supplier bill photo

Photo → Gemma-4 vision extracts “Surf Excel 1kg ×12 @110, Colgate ×24 @42” → fed to the agent → two `record_purchase_tool` calls → stock updated, itemised Hindi confirmation.



D10 — Sequence of a full voice round-trip, from spoken Hindi to spoken Hindi.

10 The Gradio app

One clean screen (`dukaan/app.py` , Gradio 6.x). Left: a chat transcript plus three inputs — **mic**, **photo**, **text** — a send button, a “speak the reply” toggle, and an autoplay audio player for the spoken answer. Right: a live **“Aaj ka hisaab”** dashboard (stock value, today’s sales, expiring soon, low stock, pending udhaar, next festival) with *Refresh* and *Alerts* buttons.

Per-session state	a `thread_id` (`gr.State`) gives each browser tab its own conversation memory
Lazy loading	Whisper / MMS / the agent load on first use (warmed in a background thread at launch) so the UI is instant
Graceful degradation	if llama-server is down the dashboard shows a banner instead of crashing; any handler error becomes a polite Hindi message

11 Deployment & operations

Per the cluster policy, GPU work goes through Slurm. `scripts/run.sbatch` runs the *whole* demo in one GPU allocation: it starts `llama-server`, polls `/health` until ready, then launches the Gradio app — and kills the server on exit.

```
sbatch scripts/run.sbatch          # llama-server + Gradio in one GPU job
uv run python -m dukaan.db --reset  # (re)build + seed both databases
uv run pytest -q                    # 31 tests
# quick debug (no Slurm):
bash scripts/serve_llm.sh & uv run python -m dukaan.app
```

Ports	llama-server `127.0.0.1:8080` · Gradio `0.0.0.0:7860` (LAN-reachable)
Config	everything overridable via env (see `.env.example`) — model paths, ports, device, TTS engine, business thresholds
Scripts	`serve_llm.sh`, `run.sbatch`, `run_local.sh`, `download_models.sh`

A REAL SLURM BUG THIS CAUGHT In an sbatch script `${BASH_SOURCE[0]}` points at Slurm's spool copy, so a `BASH_SOURCE`-derived project root was wrong and `mkdir logs` failed. Fixed by using `$SLURM_SUBMIT_DIR`. Found only by actually running the job.

12 Testing & verification

Correctness is proven two ways: a fast automated suite, and a ground-truth agent battery against the real model.

Automated suite — 31 tests, green

test_db	schema/seed counts, the SELECT guard (allows reads, blocks writes/ injection), cross-DB JOIN
test_ops	new-vs-restock (incl. the size-token regression), sale decrement + oversell warning, udhaar math, analytics, cross-DB integrity
test_tools	every tool has name/description; query_database blocks DELETE; a write tool mutates the DB
test_numwords	Hindi number-words (Rs 528 → paanch sau atthaais rupaye)
test_smoke_e2e	live agent: a Hindi lookup returns text; a Hindi udhaar command raises the balance

Agent Q&A battery (verified vs ground truth)

Task	LLM answer	Check
Stock value (inv)	Rs 2,19,692.13	exact match
Top seller (cross-DB join)	Table Eggs, 68 units	correct
Top udhaar (txn)	Shukla ji Rs 2,540 (+next two)	correct
Item stock (inv)	Aashirvaad Atta 22 units	correct
Diagnostic	reasoned over sales+margin+stock → advice	multi-tool
Sale write	stock 22 → 17	DB verified
Restock write	17 → 57	DB verified
OCR → add	2 items → 2 purchases recorded	correct

VOICE LOOP A TTS→STT round-trip confirms the spoken path: the Hindi reply is synthesised (~12 s of audio) and Whisper transcribes it back to intelligible Hindi, numbers and all.

13 Design decisions & gotchas

The non-obvious calls that shaped the build (all recorded in `tasks/lessons.md`):

Thinking off	Gemma-4's reasoning mode blanked replies → disabled via chat-template kwargs (3.4 s → 0.2 s)
ChatOpenAI instance	pass an instance, not an `openai:` string, or deepagents uses the Responses API llama.cpp lacks
Parler → MMS	Indic Parler-TTS is a gated HF repo (the machine's token isn't authorised) → defaulted to open MMS-TTS
Gradio 6.16	`gr.Chatbot` dropped `type="messages"`; `theme` moved to `launch()`
Two-DB ATTACH	physical split + a read-only attached connection keeps cross-DB analytics working
Slurm cwd	use `$SLURM_SUBMIT_DIR`, not `$BASH_SOURCE`, inside sbatch
Subagent discipline	a build subagent over-verified (ran the full test suite) and stalled a pipeline → scope build prompts tightly

14 Project layout

```
small_build_hackathon/
├── dukaan/                     # application package (~3,400 LOC)
│   ├── config.py               # env-driven config (model paths, ports, thresholds)
│   ├── db.py                   # TWO-DB layer: schemas, ATTACH reads, SELECT guard, loader
│   ├── ops.py                  # business ops + analytics (pure Python, testable)
│   ├── llm.py                  # llama-server client (chat, vision, thinking toggle)
│   ├── tools.py                # 9 LangChain tools (write + read)
│   ├── agent.py                # deepagents agent (system prompt, run_agent)
│   ├── stt.py                  # faster-whisper STT (numpy, no ffmpeg)
│   ├── tts.py                  # MMS-TTS (+ parler fallback) + markdown/number cleanup
│   ├── numwords.py             # digits/Rs → Hindi number words (for TTS)
│   ├── normalize.py            # Hindi normalize + Gemma-4 vision OCR
│   ├── proactive.py            # expiry / festival / udhaar agents + 2026 calendar
│   ├── app.py                  # Gradio single-screen app
│   ├── seed_inventory.py       # research-grounded catalog (160 SKUs + restocks)
│   └── seed_ledger.py          # research-grounded customers + ~6k sales + udhaar
├── scripts/  run.sbatch · serve_llm.sh · run_local.sh · download_models.sh
├── tests/    test_db · test_ops · test_tools · test_numwords · test_smoke_e2e (31 tests)
├── data/     inventory.db · transactions.db
├── models/   gemma-4 GGUF + mmproj   ·   vendor/  llama.cpp (CUDA build)
└── docs/     design.md · this report
```

15 Limitations & future work

- **TTS naturalness.** MMS is robotic; swapping in Indic Parler-TTS (Apache, gated) once access is granted would lift the spoken quality — `tts.py` already supports it.
- **Reminder sending.** Udhaar reminders are *drafted*, not sent; wiring a WhatsApp/SMS provider is the obvious next step.
- **Concurrency.** The demo is single-process (Gradio serialises); multi-user would want a pooled llama-server and per-request DB connections.
- **Reconciliation.** Current stock is a believable snapshot, not a strict ledger of opening + purchases − sales; real accounting would reconcile the two.
- **HF Space.** The 12B model can’t run on a free Space; a hosted deployment needs a tunnel to the cluster server or a smaller model.

16 Appendix

Key commands

```
sbatch scripts/run.sbatch                # run the whole demo (1 GPU)
uv run python -m dukaan.db --reset        # rebuild + seed both databases
uv run pytest -q                          # test suite (31)
uv run python docs/report/build_report.py  # regenerate this report
```

Example interactions (romanized)

Record stock	“das Parle-G ke packet aaye, paanch rupaye wala”
Record a sale	“do Tata Salt bik gaye”
Add udhaar	“Sharma ji ne do sau ka udhaar liya, doodh aur biscuit ka”
Lookup	“aaj kitni bikri hui?” · “kiska kitna udhaar baaki hai?”
Diagnostic	“Maggi kyun nahi bik raha?”

Configuration (selected env vars)

DUKAAN_LLM_BASE_URL	llama-server endpoint (default `http://127.0.0.1:8080/v1`)
DUKAAN_LLM_ENABLE_THINKING	Gemma-4 thinking mode (default `false`)
DUKAAN_TTS_ENGINE	`mms` (default) or `parler`
DUKAAN_INVENTORY_DB / _TRANSACTIONS_DB	the two SQLite file paths
DUKAAN_GRADIO_PORT / _LLM_PORT	service ports (7860 / 8080)

Generated from the live codebase and databases by `docs/report/build_report.py` — diagrams via Mermaid/ kroki, PDF via WeasyPrint.